



Université de Sherbrooke

SQL

Expressions spécialisées

SQL_03b

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

Scriptorum/Scriptorum/SQL_03b-Expressions-specialisees, version 1.0.0.a, en date du 2026-01-13

— document en cours de révision, ne pas citer à l'externe —

Plan

Introduction	3
1. Présentation	4
2. Groupement	6
3. Ordonnancement	12
4. Quantification	15
5. Contexte	22
Conclusion	28
Références	29
Définitions	30

Introduction

Le présent document a pour but de présenter une introduction du langage de définition de requêtes qu'on retrouve dans SQL.

1. Présentation

Restriction
 $R \sigma \text{ cond}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Projection
 $R \pi \{A, C\}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Jointure naturelle
 $R \bowtie S$

A	B	B	C
a1	b1	b1	c1
a2	b1	b2	c2
a3	b3	b3	c3
a4	b4	b3	c4

**Extension
(augmentation)**
 $R \alpha C:f$

A	B	A	B	C
a1	b1	a1	b1	f(a1,b1)
a2	b1	a2	b1	f(a2,b1)
a3	b3	a3	b3	f(a3,b3)
a4	b4	a4	b4	f(a4,b4)

Renommage
 $R \rho A:C$

A	C
...	...

Une requête n'est pas autre chose qu'une expression SELECT.

```
requête ::=  
  [ Contexte ]  
  SELECT opMode { * | projection-extension }  
  FROM listeDeJointures  
  [ Restriction ]  
  [ Groupement ]  
  [ OpComplémentaire ]  
  [ Ordonnancement ]  
  [ Divers ]  
  
opMode ::=  
  [ DISTINCT | ALL ]
```

2. Groupement

```
Groupement ::=  
[ GROUP BY { expression ... , } ]  
[ HAVING { condition ... , } ]
```

Exemple — Université

- Quel est le nombre d'évaluations consigné par étudiant ?

```
select matricule, count(*) as nbEvaluations  
from Resultat  
group by matricule;
```

- Quel est le nombre d'activités auxquelles un étudiant est (ou a été) inscrit ?

```
select matricule, count(distinct activite) as nbActivites  
from Resultat  
group by matricule;
```

- Quelles sont les personnes étudiantes (ou a été) inscrites à plus que trois activités ?

```
select matricule, count(distinct activite) as nbActivites  
from Resultat  
group by matricule  
having count(distinct activite) >=3;
```

- Quelle est l'activité n'ayant aucune évaluation ?

```
select sigle, count(te) as nb_evaluation
from Activite left join Resultat
  on Activite.sigle = Resultat.activite
group by sigle
having count(te) = 0;
```

- Quel est la note totale par étudiant pour chacun de ces cours ?

```
select matricule, activite, sum(note) as noteTotale  
from Resultat  
group by matricule, activite
```

3. Ordonnancement

```
Ordonnancement ::=  
  [ ORDER BY { expression [ ordre ] [ t_null ] ... , } ]  
  [ LIMIT { nombre | ALL } ] [ OFFSET nombre ]  
  
ordre ::=  
  ASC | DESC | USING opérateur  
  
t_null ::=  
  NULLS { FIRST | LAST }
```

LIMIT n : les n premiers tuples sont extraits selon l'ordre spécifié

OFFSET n : tous les tuples sont extraits sauf les n premiers selon l'ordre spécifié (le complément de LIMIT, si exclusif)

Exemple — Université

- Quelles sont les activités avec le plus d'inscriptions ?

/!\ ATTENTION de la qualification du count :-)

```
select activite, count(distinct matricule) as nb_inscription  
from Resultat  
group by activite  
order by nb_inscription desc  
limit 1;
```

4. Quantification

Un type de restriction dérivée de la logique du premier ordre.

```
restriction ::=  
  [ WHERE condition* ]
```

```
condition ::=  
  quantification_gen  
  | quantification_com  
  | expression
```

4.1. Quantification générale

```
quantification_gen ::=  
  [NOT] EXISTS (requête)
```

$\exists x. p(x)$

ensemble non vide

$0 <> (\text{SELECT count(*) FROM ensemble})$
EXISTS (ensemble)

$\neg \exists x. p(x)$

ensemble vide

$0 = (\text{SELECT count(*) FROM ensemble})$
NOT EXISTS (ensemble)

Exemple — Université

- Quels sont les étudiants qui n'ont aucune évaluation à ce jour ?

```
select *  
from Etudiant  
where not exists (  
    select 1  
    from Resultat  
    where Etudiant.matricule = Resultat.matricule  
);
```

- Quelles sont les activités n'ayant aucun étudiant à ce jour ?

```
select *  
from Activite  
where not exists (
```

```
select 1  
from Resultat  
where Resultat.activite = Activite.sigle  
);
```

4.2. Quantification de comparaison

```
quantification_com ::=  
    expression opComp [ANY | ALL] ensemble
```

```
opComp ::=  
    = | <> | < | <= | > | >=
```

```
ensemble ::=  
    ( requête ) | ( listeDeValeurs )
```

Exemple — Université

- Quelles sont les personnes étudiantes dont toutes les notes sont supérieures ou égales à 60 ?

5. Contexte

```
Contexte ::=  
  WITH [ RECURSIVE ] { requêteCont ... , }  
  
requêteCont ::=  
  nomReqCont [ ( { nomCol ... , } ) ] AS ( opDefTable )  
  
opDefTable ::=  
  requête | valeurs | insertion | miseAJour | retrait
```

Exemple — Université

- Quelles sont les résultats supérieurs ou égaux à 70. Pour chaque résultat, donner le matricule et le nom de l'étudiant, le sigle et le titre de l'activité, le type d'évaluation, sa description et la note.

```
WITH Resultats70 AS (  
    SELECT matricule,  
           activite,  
           te,  
           note  
    FROM Resultat  
    WHERE note >= 70)  
SELECT E.matricule,  
       E.nom,  
       A.sigle,  
       A.titre,  
       R.note  
FROM Resultats70 AS R  
JOIN Etudiant AS E  
    ON R.matricule = E.matricule  
JOIN Activite AS A  
    ON R.activite = A.sigle  
JOIN TypeEvaluation AS T
```

```
ON R.te = T.code;
```

Exemple — Université

- Quelles personnes étudiantes ne sont inscrites à aucune activité en 2013 ?

```
-- (Résultat  $\sigma$  ( $20131 \leq \text{trimestre} \leq 20133$ ))  $\pi$  {matricule}
with Inscrit2013 as (
  select distinct matricule
  from Resultat
  where trimestre between '20131' and '20133'
),
-- Étudiant - (Étudiant  $\bowtie$  Inscrits2013)  $\pi$  {matricule}
NonInscrit2013 as (
  select *
  from Etudiant
  except
  select *
  from Etudiant
  join Inscrit2013 using (matricule)
)
```

```
select *  
from NonInscrit2013;
```

- Quelles sont les notes inférieures à la moyenne pour l'activité IFT187 ?
Donner le matricule, le nom de la personne, la note et la différence avec la moyenne.

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ> ^{ςb}
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 1. Exemple — Université

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-29 19:34:24 UTC



Université de Sherbrooke